

COMP 3300 – Fall 2025  
Assignment 5, 10 pts.  
Due: Sunday, Sep 21<sup>st</sup> at 11:55p

### Objectives

- Create a GUI application using legacy code as a partial model.
- Populate a listbox in a Windows Forms application.
- Create and display a custom dialog.

### Notes

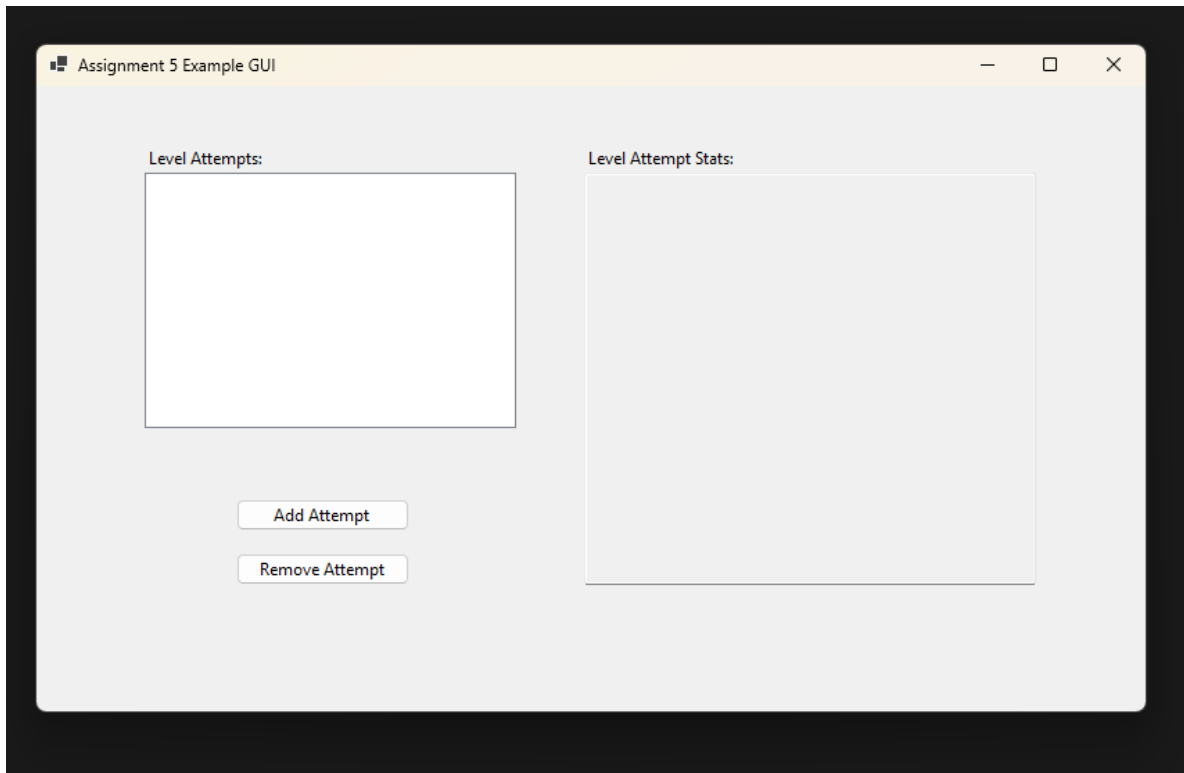
- Please read through this entire document before you begin, so you know all the requirements for the program at the outset.
- At this point you should be comfortable with reading/viewing warnings and errors in Visual Studio. From this point forward any warnings present within the console will when grading will result in lost points. ReSharper gives pointers in the form of warnings to correct issues found.
- **Any program that does not compile will receive a zero (0). Partial credit is not possible for any program that does not compile.**

### Getting Started

1. Before you start make sure you have reviewed the slides for the week and watched the provided videos located within the slides.
2. This assignment will use some code to both model and serve as an example of how some parts will function from your Assignment 2 submission, so you may want to refer to that submission as you complete this assignment. Keep in mind, this is not intended to be a complete copy of Assignment 2, rather a continuation of how some assignment 2 might evolve as we work with code.
3. Start Visual Studio 2022 and create a new C# Windows Forms App (.NET Framework).
  - a. Name the project *FirstnameLastnameA5*, e.g., SallieMaeA5.
4. Change the form title bar to be *Firstname Lastname Assignment 5*.
5. The default form should be named appropriately (based on what it is). Do not use the default Form1 name given by Visual Studio.

## Requirements

1. Add controls so that the main form looks like the following example:



- a. The GUI does not need to be a complete mirror of the above but should be close to the general design.
  - i. Your control size/widths may differ, but you should strive to ensure that the deliverable has a user-friendly design.
- b. The following are controls are required:
  - i. The control used for level attempts must be a ListBox.
  - ii. The control for level attempt stats must be a read only TextBox.

2. Create a *View* namespace within this namespace create a new form that allows you to add one that looks like the following:

The image displays two side-by-side screenshots of a Windows application window titled "Add Level Attempt". The window contains a form with five input fields and two buttons. In the left screenshot, the fields are empty except for the "Attempt Date" which is pre-filled with "9/10/2023". In the right screenshot, the fields contain the following values: "First Name" is "Tom", "Last Name" is "Sample", "Score" is "15", "Attempt Time" is "120.0", and "Attempt Date" is "9/10/2023". The "Score" field is a numeric spinner control. Both screenshots show "Add" and "Cancel" buttons at the bottom of the form.

- a. You are required to have to controls show above with the following specifications:
  - i. First/Last Name are required to allow entry of names with appropriate error checking (No null/empty/whitespace values)
  - ii. Score is a numeric (int) and allows from 0 – 100. In addition, it must use the up/down increment buttons and a user is not allowed to manually enter a number in the box.
  - iii. Time allows for decimal places (1 decimal place is enough) and should not be empty/null/whitespace. In addition, it should allow between 0 exclusive, and 120.0 inclusive.
  - iv. Date should not allow user entry but will default to the current date show in the format above. That means the date will change based on when the form is opened.
  - v. All controls, excluding labels, will display their values in the center of the control.
- b. The Add and Cancel buttons should be set as the *Accept* and *Cancel* buttons for the form.
  - i. Cancel will just close the form (the add attempt form), nothing else performed.
- c. Add functionality to this form that will read the data from the controls and make an Attempt object when the Add button is invoked.
  - i. You will need to create the class based on the data from this UI example.
  - ii. Think about and pick appropriate datatypes for these elements.
  - iii. When thinking about the date, consider other options rather than simply string.
  - iv. The class created should be located within the namespace defined below in item 3 of the instructions, so you may want to create the namespace now rather than in step 3.
- d. If there is missing or incorrect data, the user should be notified with a MessageBox.

- e. Make sure the UI has a layout and tab order that follows best practices.
3. Create a `Model` namespace and an `Attempt` and `LevelAttempts` class. These classes will be like the classes created in Assignment 2.
- a. `Attempt` is mentioned in the previous section, so you may/may not have already created the namespace/class for attempt already.
  - b. Add the following constructor to the `Attempt` class:
    - i. There should **NOT** be a default constructor.
    - ii. A five-parameter constructor that accepts the first and last name, score, time, and date as parameters and sets the corresponding properties.
      - 1. There should be appropriate precondition checks based on the requirements mentioned in the previous section.
    - iii. Add a `ToString` method to the `Attempt` class that returns the following string:
 

```
LastName, FirstName, Score, Time
```
  - c. The `LevelAttempts` class should have the following properties:
    - i. Auto-implemented properties:
      - 1. `LvlAttempts : List<Attempts>` - read only.
    - ii. Expression-bodied property:
      - 1. `Count` which returns the number of `Attempt` objects present within the list.
  - d. Add the following constructor to the `Attempts` class:
    - i. Add a default constructor that just instantiates the `LvlAttempts` property object.
  - e. Add an `Add` method to the `LevelAttempts` class that accepts an `Attempt` object as a parameter and then adds the attempt to the collection of `Attempt`. This method should not return anything.
  - f. Overload the `Add` method with another one that accepts the parameters for creating an `Attempt` object. Create the `Attempt` object based on the data passed in and add it to the collection of `Attempt` objects.
  - g. Add a `Remove` method that accepts an `Attempt` object and removes the object from the roster. The method should return a `Boolean` to indicate whether the attempt was removed from the collection.

- h. Add Attempt Stats (like Assignment 2):
  - i. A method that finds and returns the minimum score for the attempts in the collection.
  - ii. A method that finds and returns the maximum score for the attempts in the collection.
  - iii. A method that finds and returns the average time for the attempts in the collection.
  - iv. A method that finds and returns the score range for the attempts in the course.
- 4. Add functionality so the main form functions as follows:
  - a. It has a `private` field for `LevelAttempts`.
  - b. When the Add button is invoked it should display the form that allows the Attempt information to be entered. Upon returning from said form via the Add button, the `Attempt` needs to be added to the `LevelAttempts` collection and the `ListBox`, which shows the list of attempts in the collection, should be updated as well as the Stats output. No action should be taken if `Cancel` was invoked.
    - i. The summary output should be formatted as shown below with the average formatted to two decimal places.

Assignment 5 Example GUI

Level Attempts:

Smith, Kim, 10, 62.53  
Smart, Roger, 5, 25.78  
Jones, Bob, 35, 93.2  
Doe, Joe, 1, 112.78

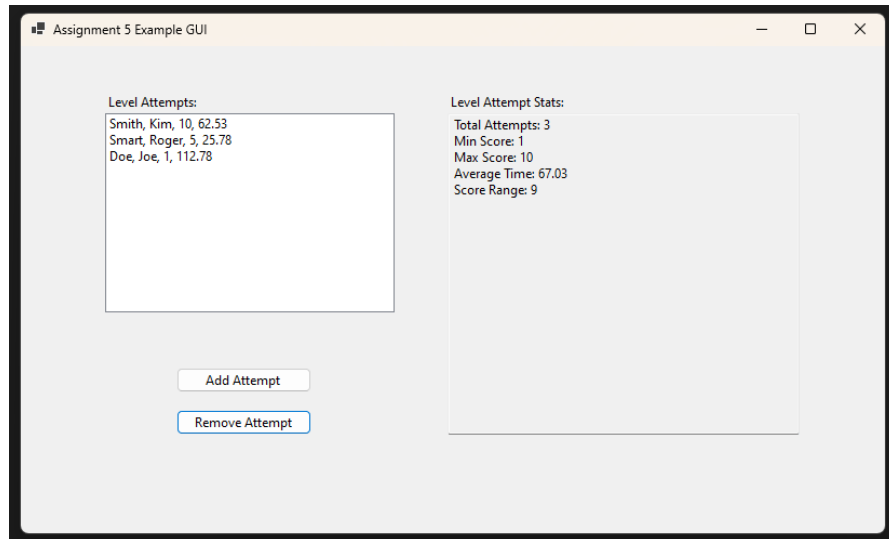
Add Attempt

Remove Attempt

Level Attempt Stats:

Total Attempts: 4  
Min Score: 1  
Max Score: 35  
Average Time: 73.57  
Score Range: 34

- c. When the Remove Attempt button is invoked, it should get the currently selected item in the list box and then remove that Attempt from the attempts and then update the list box and summary stats output accordingly.
- i. *Hint: The SelectedItem property gets the actual object that is selected in the list box. Depending on how you generate the items in the ListBox, you may be able to use this – there are multiple ways to accomplish this.*



## Documentation and coding standards

1. For all the class headers and the public properties and methods, add the XML documentation header and complete it.
2. Naming elements in our programs is important, they should be descriptive and clear what they do. Do not rely on the default generated names.

## Grading breakdown

- 10 pts. – Exercise completed successfully with at most only a minor error and follows best programming practices.
- 7 pts. – Exercise is missing some functionality or has some errors in it and mostly follows best programming practices.
- 4 pts. – Exercise is missing most of the major functionality or has major errors or best programming practices not followed.
- 0 pts. – Exercise not submitted, does not compile, or barely started.

## Submission

Make sure you clean your solution (Build → Clean Solution) and then zip up your project which needs to include the solution file into a ZIP file called *YourNameA5.zip* and submit the assignment by the due date.